NOTE This solution has one limitation: users can add only one task per timestamp. Because tasks are uniquely identified by `uid` and `tid` (the primary key) there can't be two tasks for the same user at the same time. Our timestamp comes with millisecond resolution, so it should be fine.

Using a partition key and a sort key uses two of your table's attributes. For the partition key, an unordered hash index is maintained; the sort key is kept in a sorted index for each partition key. The combination of the partition key and the sort key uniquely identifies an item if they are used as the primary key. The following data set shows the combination of unsorted partition keys and sorted sort keys:

```
["john", 1] => {            ①
  "uid": "john",
  "tid": 1,
  "description": "prepare customer presentation"
}
["john", 2] => {            ②
  "uid": "john",
  "tid": 2,
  "description": "plan holidays"
}
["emma", 1] => {            ③
  "uid": "emma",
  "tid": 1,
  "description": "prepare lunch"
}
["emma", 2] => {
  "uid": "emma",
  "tid": 2,
  "description": "buy nice flowers for mum"
}
["emma", 3] => {
  "uid": "emma",
  "tid": 3,
  "description": "prepare talk for conference"
}
```